Design a Scalable Cloud Architecture

Task 1

Pick any one app you use daily (like Zomato, BookMyShow, or Spotify) and write a short description of a new feature you would want to build for it that would require cloud hosting and scaling.

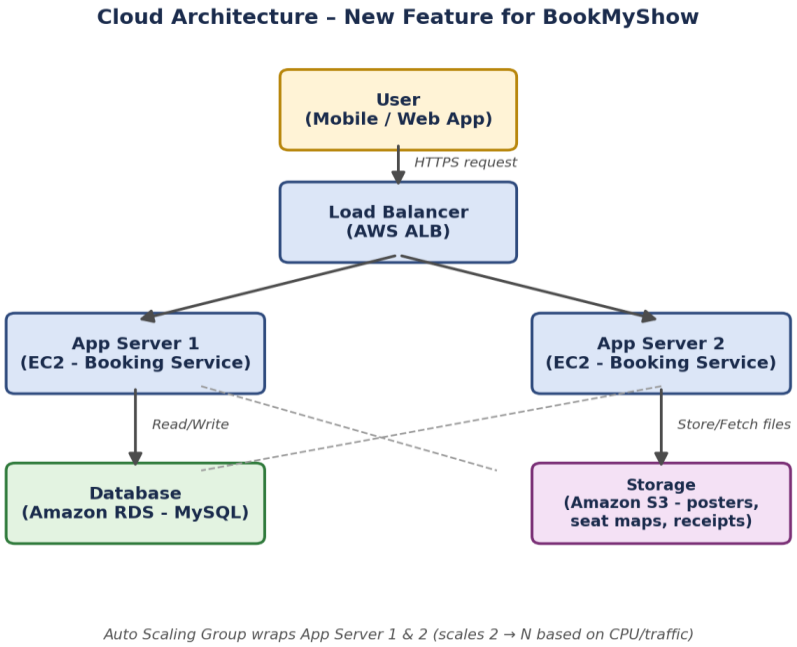
App chosen: BookMyShow

New feature: "Group Watch Party Booking" — a feature that lets a group of friends book seats for the same movie show together in real time. One person starts a group booking session and shares a link/code; everyone in the group joins the same session, sees the seat map update live as others pick seats, and the seats get locked for a short countdown so two people can't grab the same seat. This needs cloud hosting and scaling because many independent group sessions can be active at once (especially on a big release night), each session needs near-real-time updates pushed to all its members, and seat-lock data has to stay consistent even under heavy concurrent access.

Task 2

Draw a basic cloud architecture diagram (hand-drawn or using a tool like draw.io) for your chosen feature, including at least: a load balancer, two app servers, a database, and a storage service (like AWS S3 or Azure Blob). Label each component clearly.

Diagram (created using a diagramming tool, labelled as required):



Component roles:

• User (Mobile/Web App) — sends the group-booking request over HTTPS.

• Load Balancer (AWS ALB) — receives all incoming requests and distributes them across the app servers.

• App Server 1 & App Server 2 (EC2 — Booking Service) — run the group-session and seat-locking logic; sit inside an Auto Scaling Group so more servers launch automatically as more group sessions become active.

• Database (Amazon RDS — MySQL) — stores bookings, seat status, and group-session membership.

• Storage (Amazon S3) — stores static assets such as movie posters, seat-map layouts, and generated e-ticket/receipt files.

Task 3

List three security measures you would add to your architecture to protect user data and prevent attacks. For each, explain in 1-2 lines how it helps (e.g., HTTPS, firewalls, authentication methods).

1. HTTPS/TLS encryption on all traffic — encrypts data between the user's app and the load balancer, so seat selections, payment details, and session links can't be read or tampered with if intercepted over the network.

2. Security groups + Web Application Firewall (WAF) — security groups restrict which ports/IPs can reach the app servers and database directly, while a WAF sitting in front of the load balancer blocks common attacks such as SQL injection and bot-driven seat-scalping attempts during a rush.

3. Authentication with short-lived tokens (OAuth/JWT) and rate limiting — every group member must be logged in and hold a valid token before they can lock a seat, preventing strangers from joining a private group session; rate limiting stops a single user or script from spamming seat-lock requests and starving other genuine users.

Task 4

Write a short document (200-300 words) explaining how your architecture would handle a sudden spike in users during a big event (like IPL finals for BookMyShow or a major album release on Spotify). Describe which parts would scale and how.

When a big release drops on BookMyShow — for example, tickets opening for a much-awaited blockbuster — thousands of users open the app within the same few minutes to start group bookings. My architecture is designed so that different layers absorb this spike in different ways instead of everything hitting one server.

The Load Balancer is the first line of defence: it simply spreads incoming requests evenly across however many app servers are currently running, so no single server gets overwhelmed. The App Servers sit inside an Auto Scaling Group with a target-tracking policy based on CPU utilization — if average CPU across the servers stays above roughly 60% for a few minutes (as configured in the Auto-Scaling task earlier), the group automatically launches additional EC2 instances, going from the usual 2 servers up to as many as the maximum allows, and the load balancer starts routing new group-session traffic to them within moments. Because the booking/session logic is stateless on each server (session and seat-lock data lives in the database, not on the server itself), any new instance can immediately start handling real traffic without needing a warm-up dataset.

The Database is the part that needs the most care during a spike, since every seat-lock check hits it. To handle this, I would add a read replica so read-heavy operations (like refreshing the live seat map) don't compete with write operations (locking a seat), and use connection pooling so a burst of app-server instances doesn't exhaust database connections. The Storage layer (S3) barely feels the spike since posters and seat-map images are mostly read-only and can be cached at the edge (CDN), so it scales almost for free. Once the release rush settles, the Auto Scaling Group scales back down to the minimum, and the read replica can be scaled down too, keeping costs low outside of peak events.

Task 5

Present your architecture design to a friend or family member, ask them for one piece of feedback or a question, and write down what they said along with your response.

Who I presented to: my cousin (not from a tech background)

I walked her through the diagram using the BookMyShow group-booking idea — explaining it like "when your request comes in, a load balancer is like a manager at a counter who decides which of the two clerks (app servers) should attend to you, so no one clerk gets overloaded. The clerk stores your seat booking in a database, and posters/tickets are kept in separate cloud storage."

Her feedback/question:

"What happens if the one 'manager' — the load balancer — itself goes down? Doesn't the whole system stop working then, even if you have extra clerks?"

My response:

That's a great catch — I explained that in a real AWS setup, the load balancer isn't actually a single machine either; AWS runs it across multiple availability zones behind the scenes, so if one copy fails, another takes over automatically without the user noticing. I told her this is exactly why we design every layer (load balancer, app servers, database) to have more than one copy running — the whole point of cloud architecture is that no single part should be a single point of failure. She said that made it click for her why companies use cloud providers instead of just running one big server.